Multicore Processor: A multicore processor is a single CPU chip that contains two or more independent processing cores, each capable of executing instructions.

Example: Dual-core (2 cores), Quad-core (4 cores), Octa-core (8 cores)

Why did multicore systems become necessary?

Reason 1: Clock speed cannot increase forever

Increasing frequency means:

- More switching activity
- More heat generation
- More power consumption

Modern CPUs cannot run at extremely high GHz continuously due to thermal limits.

Reason 2: Diminishing returns in instruction-level parallelism

Though pipelining, superscalar execution, and branch prediction exists, but still:

- Many instructions depend on previous ones
- So parallel execution inside one core is limited

This is why superscalar CPUs can't give unlimited speedup.

Reason 3: Memory bottleneck

CPU gets faster, but memory access is slower. So, CPU often waits for RAM access and for cache misses.

Reason 4: Better performance per watt

Multicore improves:

- performance without huge increase in frequency
- efficiency

Instead of one core at 5 GHz, 4 cores at 2.5 GHz can be used

Result:

- better total throughput
- lower heat per core

Reason 5: Parallel workloads are everywhere today

Modern applications are naturally parallel:

- Web browsers (tabs, rendering, downloads)
- Video editing

- AI and ML workloads
- Gaming
- Scientific simulations
- Servers handling many users

So multicore is needed to handle:

- multiple tasks at the same time
- many users simultaneously

Reason 6: Cost-effective scaling

Instead of building one extremely powerful core, build multiple simpler cores. This gives better scalability and better use of silicon chip area.

Multicore architecture refers to the internal design and organization of a processor chip containing multiple cores, including how they share cache, memory, and communicate with each other.

Basic structure of a multicore processor:

A typical multicore CPU has:

- Multiple Cores
- Cache Memory (L1, L2, L3)
- Interconnect (connects cores + cache + memory controller)
- Main Memory Interface (RAM access)

Core: A core is like a mini-CPU inside the chip.

Each core typically has:

- Instruction Fetch unit
- Decode unit
- Execution units (ALU/FPU)
- Registers
- Pipeline
- Local caches (L1, sometimes L2)

Each core can run:

- One process or
- One/multiple threads (if multithreading exists)

Cache organization in multicore systems:

Cache	Speed	Size	Shared?
L1	Fastest	Small	Private per core
L2	Fast	Medium	Private or shared
L3	Slower than L2	Large	Usually shared

Two main multicore designs:

Private cache architecture: Each core has its own cache (L1+L2).

Advantages:

- Very fast access
- Less contention

Disadvantages:

- Cache coherence becomes complex
- Data duplication possible

Shared cache architecture: Multiple cores share a cache (usually L3).

Advantages:

- Better data sharing
- Less duplication
- Easier communication between cores

Disadvantages:

- Contention (many cores accessing same cache)
- Slower than private cache

Interconnection inside multicore chips:

Cores must communicate with:

- shared cache
- memory controller
- other cores

This is done using an on-chip interconnect, such as:

- Bus (simple, limited scalability)
- Ring interconnect (common in Intel CPUs)
- Mesh interconnect (used in many-core CPUs)

Memory controller and RAM access:

Modern CPUs often include an integrated memory controller. It handles communication between CPU and RAM

Why important?

- Faster memory access
- Reduced latency

Key challenges in multicore architecture:

Cache coherence: When two cores modify shared data, caches must stay consistent.

Memory contention: Multiple cores compete for RAM bandwidth.

Synchronization overhead: Threads need locks, barriers etc.

Homogeneous multicore architecture: A homogeneous multicore processor contains multiple identical cores, where each core has the same architecture and performance capability.

- Same core design
- Same instruction set
- Same speed capability

Advantages:

- Easy scheduling: OS can assign tasks to any core easily
- Balanced performance: No confusion about "which core is stronger"
- Simpler design: Easier to manufacture and optimize
- Good for general purpose work: Suitable for typical computing

Disadvantages:

Less energy optimization: All cores consume similar power even for small tasks

Not best for mixed workloads: Some tasks need high performance, some tasks need low power

Heterogeneous multicore architecture: A heterogeneous multicore processor contains different types of cores, designed for different purposes such as high performance and energy efficiency.

Example: Some cores are powerful, some cores are efficient.

Advantages:

- Better power efficiency: Small cores handle light tasks using less battery
- Better performance when needed: Big cores activate for heavy computation
- Good for mixed workloads: Background tasks + heavy apps together
- Thermal management: Reduces overheating by choosing suitable cores

Disadvantages:

- Complex scheduling: OS must decide which task goes to which core
- Design complexity: More difficult to build and optimize
- Programming complexity: Performance tuning is harder

Comparison between homogeneous core and heterogeneous core:

Feature	Homogeneous Cores	Heterogeneous Cores
Core type	Same	Different
Performance	Equal	Mixed
Power efficiency	Moderate	High
Scheduling	Simple	Complex
Best for	General workloads	Mixed workloads
Example	Typical desktop CPUs	Mobile/embedded CPUs

Which is better for HPC?

HPC systems generally prefer homogeneous multicore CPUs because of predictable performance, easier load balancing and uniform parallel execution.

But modern HPC also uses heterogeneity in another form:

- CPU + GPU (heterogeneous computing)

Memory hierarchy is an arrangement of memory levels based on speed, size, and cost, where the fastest and smallest memory is closest to the CPU, and the slowest and largest memory is farthest.

Typical memory hierarchy in multicore CPUs: From fastest → slowest:

1. Registers
2. L1 Cache
3. L2 Cache
4. L3 Cache
5. Main Memory (RAM)
6. Secondary Storage (SSD/HDD)

Memory hierarchy table:

Level	Speed	Size	Cost	Location
Registers	Fastest	Very small	Highest	Inside core
L1 Cache	Very fast	Small	High	Near core
L2 Cache	Fast	Medium	Medium	Near core
L3 Cache	Slower	Large	Lower	Shared on chip
RAM	Much slower	Very large	Cheap	Outside CPU
SSD/HDD	Slowest	Huge	Cheapest	Storage

Cache organization in multicore systems: In multicore processors, caches can be:

Private cache: Each core has its own cache.

Shared cache: Multiple cores share a cache (usually L3).

Private vs Shared cache:

Feature	Private Cache	Shared Cache
Access speed	Faster	Slower than private
Contention	Less	More
Data sharing	Harder	Easier
Duplication	More	Less
Scalability	Better locally	Limited by sharing
Example	L1/L2	L3

Why L3 cache is shared?

Because it reduces duplication of data and improves inter-core communication.

Cache coherence ensures that all cores see the same correct value of shared data, even if they have cached copies.

A **thread** is the smallest unit of execution inside a process, having its own program counter, registers, and stack, but sharing the process's memory and resources.

Why multithreading is needed in multicore systems?

If a CPU has multiple cores, running multiple threads in parallel (one per core), we can make use of the architecture.

Benefits of multithreading:

- Better CPU utilization
- Faster program execution
- Better responsiveness
- Higher throughput

Multicore + Multithreading: How they work together?

- Multicore provides hardware parallelism
- Multithreading provides software parallelism

Types of hardware multithreading:

Coarse-grained multithreading: Switches thread only when long stall happens Example: Cache miss delay

- Less switching overhead
- Less responsive switching

Fine-grained multithreading: Switches threads every cycle (or frequently)

- Hides latency better

- Higher complexity

Simultaneous Multithreading (SMT): SMT allows multiple threads to execute simultaneously within the same core by utilizing multiple execution units. Example: Intel Hyper-Threading

A CPU with 4 cores and 2 threads per core acts as 8 logical processors.

Multithreading challenges:

Race condition: Two threads update the same data, produces wrong result.

Deadlock: Two threads wait forever for each other.

Synchronization overhead: Locks for synchronization slow down performance.

False sharing: Two threads modify different variables in same cache line, as a result performance drops.

Intel x86 refers to a family of processors that implement the x86 instruction set architecture (ISA), widely used in desktops, laptops, servers, and HPC systems. Examples: Intel Core i5 / i7, Intel Xeon (servers/HPC)

A modern Intel multicore CPU generally includes multiple CPU cores, private L1 cache per core, private L2 cache per core, Shared L3 cache (Last level cache), Integrated Memory Controller (IMC), Interconnection (ring / bus / Mesh), Support for SMT (Hyper-Threading). Each Intel core contains instruction fetch unit, decode unit, execution units (ALU, FPU, SIMD units), registers, pipeline and branch prediction hardware.

Intel Hyper-Threading is Intel's implementation of Simultaneous Multithreading (SMT), where one physical core appears as two logical cores.

Why hyper-threading helps?

Sometimes one thread stalls due to cache miss, memory access delay. The other thread uses the execution units. Thus the CPU stays busy.

Intel multicore performance challenges:

Cache coherence overhead: Keeping caches consistent across cores is complex.

Memory bandwidth bottleneck: Many cores compete for RAM access.

Contention in shared L3 cache: Many cores using the same shared cache may slow access.